

---

## 8      Sorting in Linear Time

We have now seen a handful of algorithms that can sort  $n$  numbers in  $O(n \lg n)$  time. Whereas merge sort and heapsort achieve this upper bound in the worst case, quicksort achieves it on average. Moreover, for each of these algorithms, we can produce a sequence of  $n$  input numbers that causes the algorithm to run in  $\Omega(n \lg n)$  time.

These algorithms share an interesting property: *the sorted order they determine is based only on comparisons between the input elements*. We call such sorting algorithms **comparison sorts**. All the sorting algorithms introduced thus far are comparison sorts.

In Section 8.1, we'll prove that any comparison sort must make  $\Omega(n \lg n)$  comparisons in the worst case to sort  $n$  elements. Thus, merge sort and heapsort are asymptotically optimal, and no comparison sort exists that is faster by more than a constant factor.

Sections 8.2, 8.3, and 8.4 examine three sorting algorithms—counting sort, radix sort, and bucket sort—that run in linear time on certain types of inputs. Of course, these algorithms use operations other than comparisons to determine the sorted order. Consequently, the  $\Omega(n \lg n)$  lower bound does not apply to them.

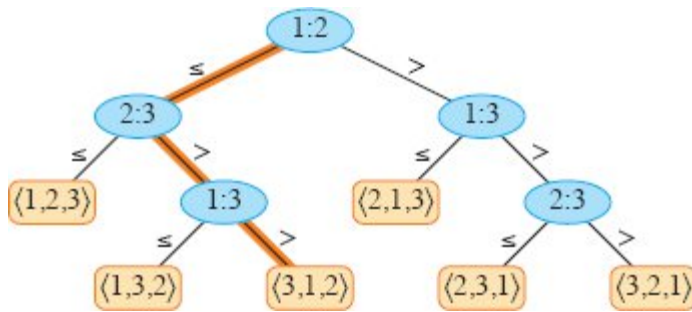
---

### 8.1      Lower bounds for sorting

A comparison sort uses only comparisons between elements to gain order information about an input sequence  $\langle a_1, a_2, \dots, a_n \rangle$ . That is, given two elements  $a_i$  and  $a_j$ , it performs one of the tests  $a_i < a_j$ ,  $a_i \leq a_j$ ,

$a_i = a_j$ ,  $a_i \geq a_j$ , or  $a_i > a_j$  to determine their relative order. It may not inspect the values of the elements or gain order information about them in any other way.

Since we are proving a lower bound, we assume without loss of generality in this section that all the input elements are distinct. After all, a lower bound for distinct elements applies when elements may or may not be distinct. Consequently, comparisons of the form  $a_i = a_j$  are useless, which means that we can assume that no comparisons for exact equality occur. Moreover, the comparisons  $a_i \leq a_j$ ,  $a_i \geq a_j$ ,  $a_i > a_j$ , and  $a_i < a_j$  are all equivalent in that they yield identical information about the relative order of  $a_i$  and  $a_j$ . We therefore assume that all comparisons have the form  $a_i \leq a_j$ .



**Figure 8.1** The decision tree for insertion sort operating on three elements. An internal node (shown in blue) annotated by  $i : j$  indicates a comparison between  $a_i$  and  $a_j$ . A leaf annotated by the permutation  $\langle \pi(1), \pi(2), \dots, \pi(n) \rangle$  indicates the ordering  $a_{\pi(1)} \leq a_{\pi(2)} \leq \dots \leq a_{\pi(n)}$ . The highlighted path indicates the decisions made when sorting the input sequence  $\langle a_1 = 6, a_2 = 8, a_3 = 5 \rangle$ . Going left from the root node, labeled  $1:2$ , indicates that  $a_1 \leq a_2$ . Going right from the node labeled  $2:3$  indicates that  $a_2 > a_3$ . Going right from the node labeled  $1:3$  indicates that  $a_1 > a_3$ . Therefore, we have the ordering  $a_3 \leq a_1 \leq a_2$ , as indicated in the leaf labeled  $\langle 3, 1, 2 \rangle$ . Because the three input elements have  $3! = 6$  possible permutations, the decision tree must have at least 6 leaves.

## The decision-tree model

We can view comparison sorts abstractly in terms of decision trees. A **decision tree** is a full binary tree (each node is either a leaf or has both children) that represents the comparisons between elements that are

performed by a particular sorting algorithm operating on an input of a given size. Control, data movement, and all other aspects of the algorithm are ignored. Figure 8.1 shows the decision tree corresponding to the insertion sort algorithm from Section 2.1 operating on an input sequence of three elements.

A decision tree has each internal node annotated by  $i : j$  for some  $i$  and  $j$  in the range  $1 \leq i, j \leq n$ , where  $n$  is the number of elements in the input sequence. We also annotate each leaf by a permutation  $\langle \pi(1), \pi(2), \dots, \pi(n) \rangle$ . (See Section C.1 for background on permutations.) Indices in the internal nodes and the leaves always refer to the original positions of the array elements at the start of the sorting algorithm. The execution of the comparison sorting algorithm corresponds to tracing a simple path from the root of the decision tree down to a leaf. Each internal node indicates a comparison  $a_i \leq a_j$ . The left subtree then dictates subsequent comparisons once we know that  $a_i \leq a_j$ , and the right subtree dictates subsequent comparisons when  $a_i > a_j$ . Arriving at a leaf, the sorting algorithm has established the ordering  $a_{\pi(1)} \leq a_{\pi(2)} \leq \dots \leq a_{\pi(n)}$ . Because any correct sorting algorithm must be able to produce each permutation of its input, each of the  $n!$  permutations on  $n$  elements must appear as at least one of the leaves of the decision tree for a comparison sort to be correct. Furthermore, each of these leaves must be reachable from the root by a downward path corresponding to an actual execution of the comparison sort. (We call such leaves “reachable.”) Thus, we consider only decision trees in which each permutation appears as a reachable leaf.

### **A lower bound for the worst case**

The length of the longest simple path from the root of a decision tree to any of its reachable leaves represents the worst-case number of comparisons that the corresponding sorting algorithm performs. Consequently, the worst-case number of comparisons for a given comparison sort algorithm equals the height of its decision tree. A lower bound on the heights of all decision trees in which each permutation appears as a reachable leaf is therefore a lower bound on

the running time of any comparison sort algorithm. The following theorem establishes such a lower bound.

***Theorem 8.1***

Any comparison sort algorithm requires  $\Omega(n \lg n)$  comparisons in the worst case.

***Proof*** From the preceding discussion, it suffices to determine the height of a decision tree in which each permutation appears as a reachable leaf. Consider a decision tree of height  $h$  with  $l$  reachable leaves corresponding to a comparison sort on  $n$  elements. Because each of the  $n!$  permutations of the input appears as one or more leaves, we have  $n! \leq l$ . Since a binary tree of height  $h$  has no more than  $2^h$  leaves, we have

$$n! \leq l \leq 2^h,$$

which, by taking logarithms, implies

$$\begin{aligned} h &\geq \lg(n!) && \text{(since the } \lg \text{ function is monotonically increasing)} \\ &= \Omega(n \lg n) && \text{(by equation (3.28) on page 67).} \end{aligned}$$

■

***Corollary 8.2***

Heapsort and merge sort are asymptotically optimal comparison sorts.

***Proof*** The  $O(n \lg n)$  upper bounds on the running times for heapsort and merge sort match the  $\Omega(n \lg n)$  worst-case lower bound from Theorem 8.1.

■

**Exercises**

***8.1-1***

What is the smallest possible depth of a leaf in a decision tree for a comparison sort?

### 8.1-2

Obtain asymptotically tight bounds on  $\lg(n!)$  without using Stirling's approximation. Instead, evaluate the summation  $\sum_{k=1}^n \lg k$  using techniques from Section A.2.

### 8.1-3

Show that there is no comparison sort whose running time is linear for at least half of the  $n!$  inputs of length  $n$ . What about a fraction of  $1/n$  of the inputs of length  $n$ ? What about a fraction  $1/2^n$ ?

### 8.1-4

You are given an  $n$ -element input sequence, and you know in advance that it is partly sorted in the following sense. Each element initially in position  $i$  such that  $i \bmod 4 = 0$  is either already in its correct position, or it is one place away from its correct position. For example, you know that after sorting, the element initially in position 12 belongs in position 11, 12, or 13. You have no advance information about the other elements, in positions  $i$  where  $i \bmod 4 \neq 0$ . Show that an  $\Omega(n \lg n)$  lower bound on comparison-based sorting still holds in this case.

---

## 8.2 Counting sort

*Counting sort* assumes that each of the  $n$  input elements is an integer in the range 0 to  $k$ , for some integer  $k$ . It runs in  $\Theta(n + k)$  time, so that when  $k = O(n)$ , counting sort runs in  $\Theta(n)$  time.

Counting sort first determines, for each input element  $x$ , the number of elements less than or equal to  $x$ . It then uses this information to place element  $x$  directly into its position in the output array. For example, if 17 elements are less than or equal to  $x$ , then  $x$  belongs in output position 17. We must modify this scheme slightly to handle the situation in which several elements have the same value, since we do not want them all to end up in the same position.

The COUNTING-SORT procedure on the facing page takes as input an array  $A[1 : n]$ , the size  $n$  of this array, and the limit  $k$  on the